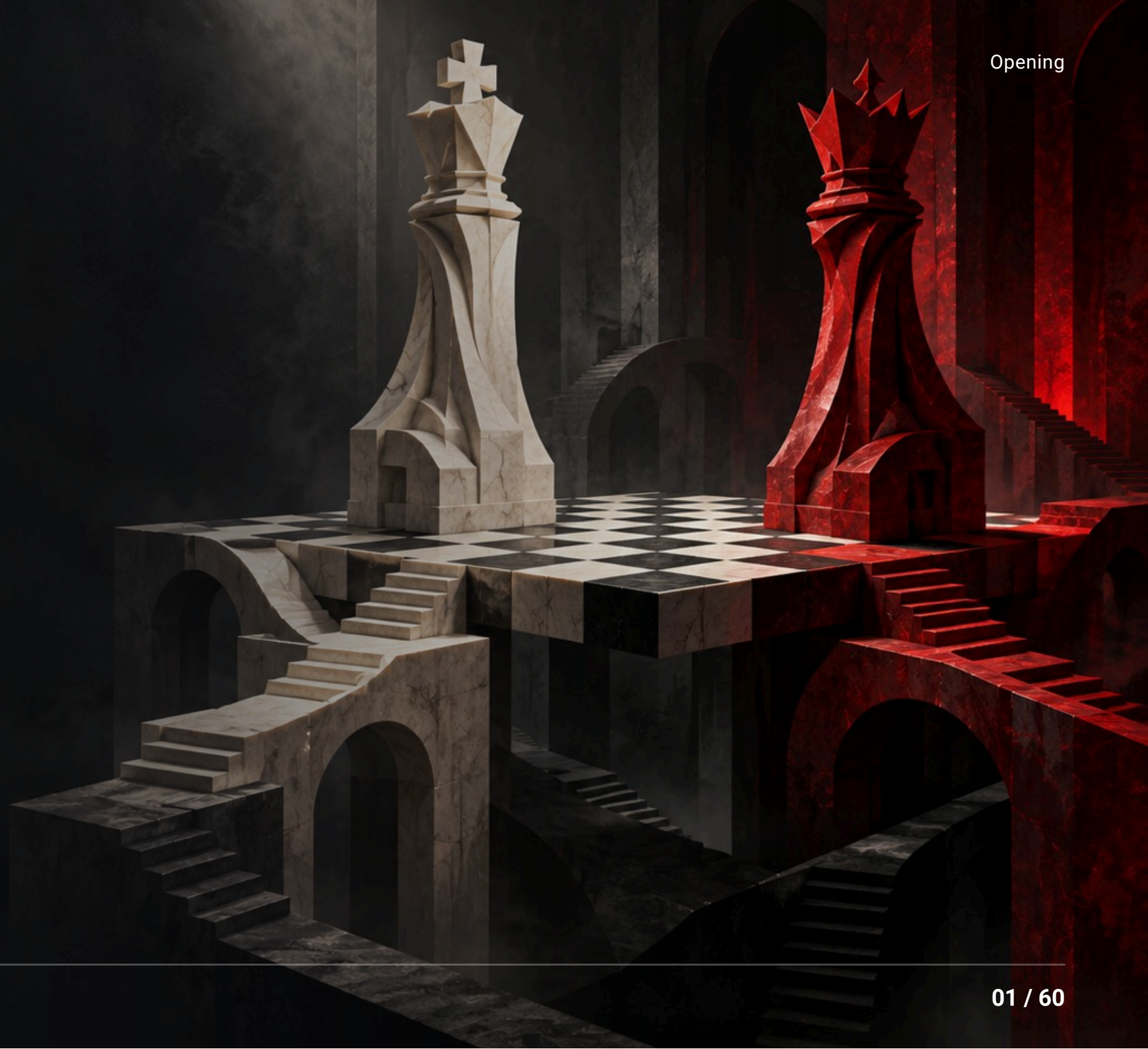


Adversarial search

Unit 5 · Intelligent Systems

School of Computer Science · Ciudad Real
Universidad de Castilla-La Mancha

Rubén Juárez Cádiz



A decision must survive the opponent's reply

60 slides · Five laboratories · Worked solutions

01

Model

Identify the player to move and the terminal outcome.

02

→ **Back up**

Propagate MAX and MIN values through the game tree.

03

→ **Reduce work**

Use bounds, move ordering and repeated positions.

04

→ **Extend**

Add estimates, chance or hidden information.

Explain the selected action, the search evidence and the limits of the opponent model.

A puzzle asks for a path; a game asks for decisions

Aspect	Loopover / single-agent search	Adversarial game
Who acts?	One planning agent.	Players take turns.
Outcome	Reach a goal, perhaps at minimum cost.	Obtain a terminal utility.
Returned object	A sequence of legal actions.	A move now and decisions for later states.
Uncertainty about continuation	Which path should the solver explore?	Which response will the opponent choose?

State, actions and transition functions remain useful. Control over future actions changes.

The classical minimax model has explicit assumptions

Two players

MAX and MIN alternate turns in this unit's examples.

Zero-sum utility

Every value uses MAX's perspective; MIN prefers lower values.

**Deterministic
transitions**

A legal action has one specified successor.

Perfect information

The current game state is observable to both players.

Full-tree guarantees also require a finite game tree or correct handling of repetitions and draws.

The game definition makes each search operation precise

Operation	Meaning
Initial state	A legal starting position, including player to move.
Player(s)	Which player chooses the next action.
Actions(s)	Exactly the legal moves available in s.
Result(s,a)	The complete successor after applying a.
Terminal(s)	Whether play has ended under the game rules.
Utility(s)	The exact outcome for MAX at a terminal state.

A board diagram may be only part of the state

Position and turn

The same arrangement can offer different legal actions depending on who moves next.

Relevant history

Repetition rules or special-move rights may distinguish otherwise identical boards.

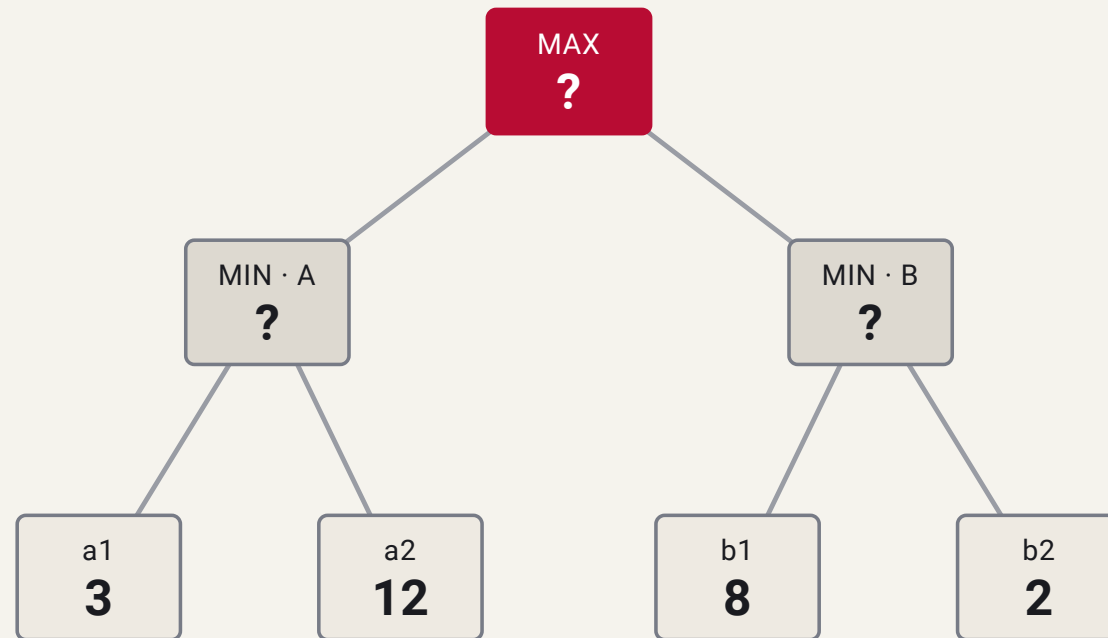
A useful invariant

State equality must imply the same legal continuations and outcome rules.

A cache key must preserve those distinctions.

Merge positions only when they represent the same decision problem.

One ply is one player's action



Root

The current decision belongs to MAX.

Next level

MIN selects a response to A or B.

Leaves

Terminal utilities here: 3, 12, 8, 2.

Every number uses the same player’s perspective

Outcome	One possible utility scale for MAX
MAX wins	+1
Draw	0
MIN wins	−1

At MIN nodes, choose the smallest MAX utility. Do not change its sign merely because it is MIN’s turn.

The source board already contains a win

X	O	
	X	
O		X

X occupies the main diagonal

Squares 1, 5 and 9 form a completed line.

Use the terminal rule

Return the exact X-win utility.

In the laboratories: **+100**.

Check Terminal(s) before a depth cutoff or a heuristic evaluation.

The backup rule follows the player who controls the node

Terminal

$$V(s) = U(s).$$

MAX to move

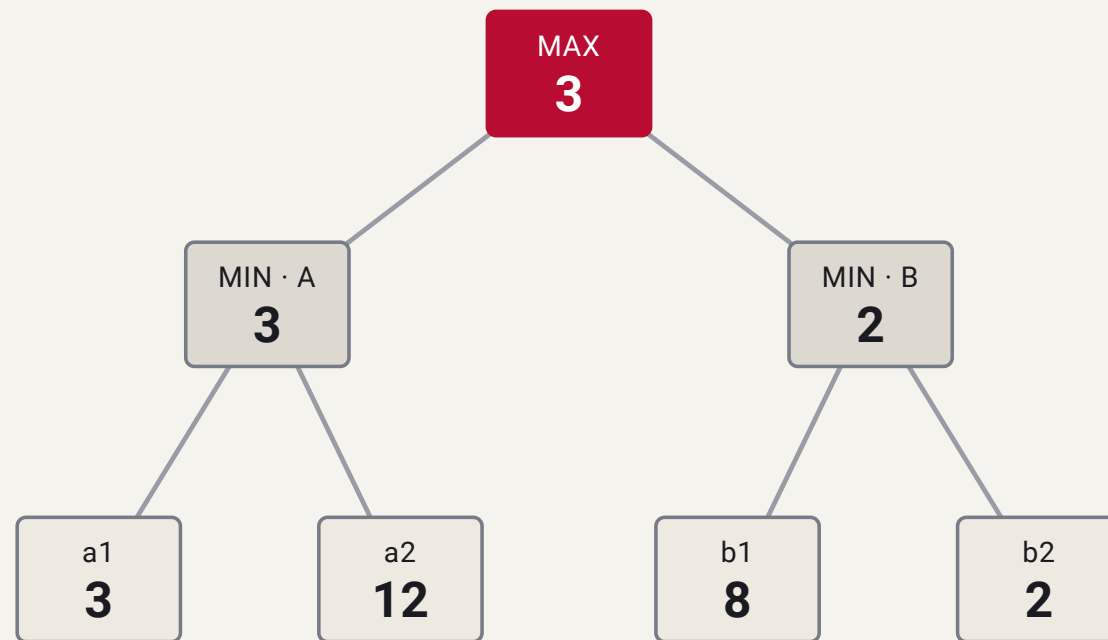
$$V(s) = \text{maximum } V(\text{child}) \text{ over legal actions.}$$

MIN to move

$$V(s) = \text{minimum } V(\text{child}) \text{ over legal actions.}$$

The value describes optimal play in the specified game, not the outcome of every possible continuation.

The four-leaf example backs up to 3



Evaluate the MIN nodes

$$A = \min(3, 12) = 3$$

$$B = \min(8, 2) = 2$$

Then evaluate MAX

$$V(\text{root}) = \max(3, 2) = 3.$$

Choose action A.

A large reachable payoff may be unavailable against MIN

12 is reachable.

3 is guaranteed.

It appears on a legal continuation below A.

MIN can choose 3 instead, so MAX cannot force 12.

The opponent's choice changes how values combine. An optimistic maximum over all leaves is incorrect.

A recursive value function and a root action

```
VALUE(s):  
    if Terminal(s): return Utility(s)  
    values ← [VALUE(Result(s,a)) for a in Actions(s)]  
    if Player(s) == MAX: return max(values)  
    else: return min(values)  
  
DECIDE(s):  
    if Terminal(s): return NO_MOVE  
    score[a] ← VALUE(Result(s,a)) for each legal action a  
    return first action with best score for Player(s)
```

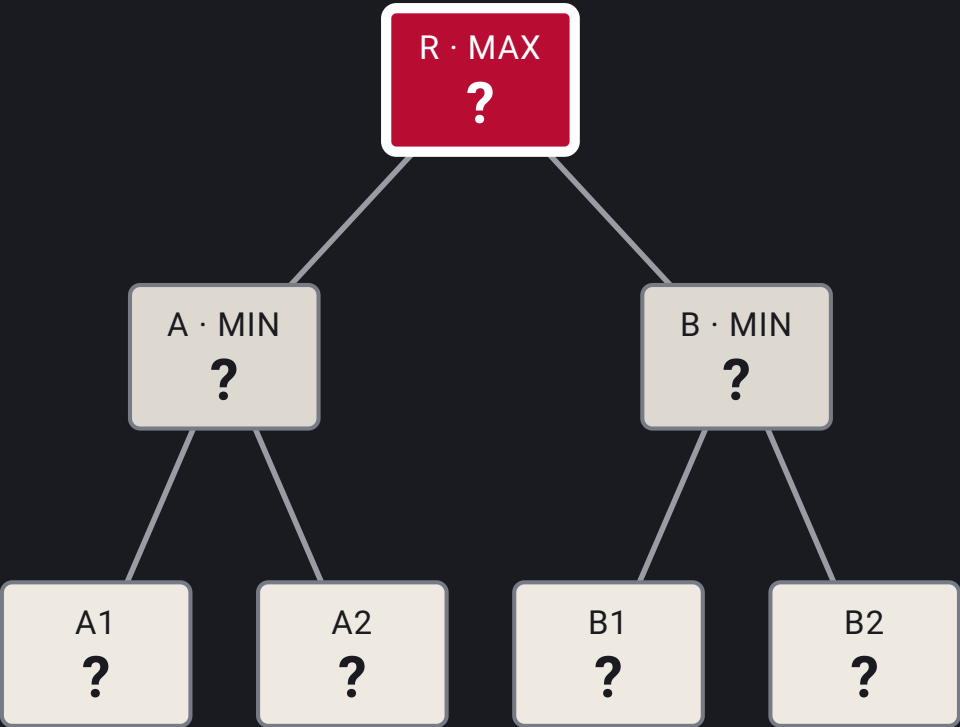
A deterministic tie rule makes examples reproducible.

Value, first move and strategy answer different questions

Object	What it tells us
Position value	The outcome obtainable under optimal play.
Root action	The next legal move selected by the search.
Principal variation	One optimal line under a particular tie policy.
Strategy	A conditional action choice for relevant future decision states.

After the opponent moves, the program can search the observed state again.

A visible trace from leaves to the root



Leaf values

Source: 3, 12, 8, 2

Algorithm

Minimax

Back

Next event

To result

Reset

Enter R (MAX).

Leaves read

0/4

Window

$[-\infty, \infty]$

Event 1/20

Read left to right. Bright outline = current node. Dashed fade = skipped. A bound such as ≤ 5 is not an exact node value.

The guarantee belongs to a precisely stated model

Full finite search

With legal transitions, exact terminal utilities and exhaustive backups, minimax finds an optimal action.

What the result assumes

The game is modelled correctly and the opponent can choose its best response.

A cutoff estimate changes the guarantee.

“Optimal against optimal opposition” does not mean every real opponent behaves that way.

The tree grows exponentially with depth

$$O(b^m)$$

Time for a full depth- m tree with branching factor bounded by b .

Depth-first storage

$O(bm)$ when each stack level retains its generated successors.

Lazy generation can reduce traversal storage toward $O(m)$, excluding state and cache costs.

A search budget makes leaf estimates necessary in large games.

A depth cutoff changes what a leaf value means

Leaf reached	Returned value	Interpretation
Terminal position	Utility(s)	Exact game outcome.
Nonterminal at depth 0	Eval(s)	Estimated continuation value.
Nonterminal above the cutoff	MAX/MIN backup	Optimal for the searched tree and its leaf scores.

Cutoff minimax can be exact for its truncated tree while choosing a suboptimal move in the full game.

Terminal outcomes take priority over the cutoff

```
VALUE(s, depth):  
    if Terminal(s): return Utility(s)  
    if depth == 0: return Eval(s)  
    values ← [VALUE(Result(s,a), depth-1)  
              for a in Actions(s)]  
    return max(values) if Player(s)==MAX else min(values)
```

A win found exactly at the depth limit must retain its true terminal utility.

Features describe advantages from MAX’s perspective

Feature difference	Precise definition in our tic-tac-toe lab
$O_x - O_o$	Open lines for X minus open lines for O. An open line contains no opponent mark.
$T_x - T_o$	Immediate threats: exactly two own marks and one empty square.
C	+1 for an X in the centre, -1 for O, otherwise 0.

Eval = clamp($w_o(O_x - O_o) + w_t(T_x - T_o) + 2C$, -90, 90).

A promising feature score can miss a forced win

1	2	3
4	5	6
O	8	X

X to move

O occupies square 7.

X occupies square 9.

Default one-ply search

Square 5 scores **21**.

Square 3 scores **16**.

The full game favours square 3.

The centre looks attractive to this evaluation function, but choosing it gives up a forced win.

Search depth and feature weights change the choice

Position

Missed win · X to move

1	2	3
4	5	6
0	8	X

Current features: open X/O 5/5; threats X/O 0/0; centre 0.

Reset experiment

Depth: 1

Open-line weight: 3

Threat weight: 10

Square	Search score	Exact move value
1	16	100
2	6	0
3	16	100
4	6	0
5 ← chosen	21	0
6	13	100
8	3	-100

Choose square 5. Search score 21; exact value after this move 0.

Terminal utility: X +100, draw 0, O -100. Equal search scores use the lowest square number. "Exact" is a full-search reference.

Deeper search recovers the winning action in this case

Search on O7 / X9	Selected square	Search score	True value after the move
One ply, default weights	5	21, an estimate	0: a draw with optimal continuation.
Two plies, default weights	3	0, an estimate	+100: X can force a win.
Full search to terminal states	1, first best move	+100, exact	+100.

A search score of 0 and an exact value of 0 make different claims.

A cutoff can hide the consequence of a move

Before the horizon

A position looks strong because of material, space or threats.

Just beyond it

The opponent has a forcing reply that changes the assessment.

Useful responses

Increase depth where feasible.

Improve tactical features.

Extend selected unstable positions using a justified rule.

A heuristic game evaluator does not need the admissibility or edge-consistency condition used by A*.

Quiescence search extends selected tactical positions

01

Reach cutoff

The nominal depth limit has been reached.

02

→ Check stability

A domain rule identifies forcing tactical continuations.

03

→ Extend selectively

Search those moves before evaluating the position.

The extension policy changes the searched tree and must have a termination rule.

A proven alternative can make a subtree irrelevant

MAX already has value 5

An earlier root action guarantees that score.

A new MIN branch reaches 2

MIN can hold this branch to at most 2, whatever the remaining leaves contain.

MAX can retain 5 without determining the exact value of the inferior branch.

The search carries a window of relevant values

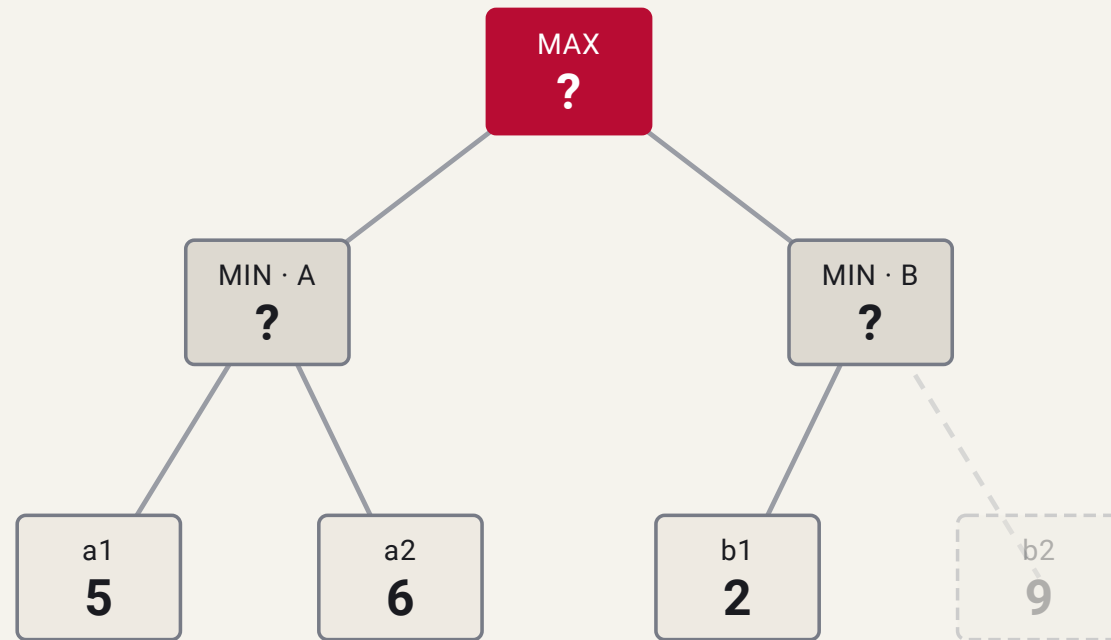
α : lower threshold An alternative already available to MAX along the current search path.

β : upper threshold An alternative already available to MIN along the current search path.

Cutoff Stop remaining children when $\alpha \geq \beta$.

Bounds are relative to the current search context. They are not exact values of every descendant.

Only three leaves are needed here



Read left to right

$A = \min(5,6) = 5$.

Root α becomes 5.

Enter MIN branch B

First leaf: 2, so β becomes 2.

$\beta \leq \alpha$: skip leaf 9.

Root keeps A with value 5.

The cutoff establishes $B \leq 2$. The search did not need to learn whether B equals 2.

One recursive function with bounds

```
AB(s, depth,  $\alpha$ ,  $\beta$ ):  
  if Terminal(s): return Utility(s)  
  if depth == 0: return Eval(s)  
  v  $\leftarrow$  -infinity if Player(s)==MAX else +infinity  
  for a in ordered_actions(s):  
    child  $\leftarrow$  AB(Result(s,a), depth-1,  $\alpha$ ,  $\beta$ )  
    if Player(s)==MAX:  
      v  $\leftarrow$  max(v, child);  $\alpha \leftarrow$  max( $\alpha$ , v)  
    else:  
      v  $\leftarrow$  min(v, child);  $\beta \leftarrow$  min( $\beta$ , v)  
    if  $\alpha \geq \beta$ : break  
  return v
```

At the root, record an action only on strict improvement of its current best score.

A returned number can be exact, a lower bound or an upper bound

Evidence from a search call	Meaning for that subtree
A MAX cutoff at value v	The subtree value is at least v.
A MIN cutoff at value v	The subtree value is at most v.
A correctly completed full-window search	The returned minimax root value is exact for that searched tree.

A cache entry must preserve whether the number is EXACT, LOWER or UPPER.

An equal bound is not proof of an equal action value

Search step	What is known
A has leaves 5 and 6	$A=5$ exactly, so root $\alpha=5$.
B first returns a leaf value 5	$B \leq 5$. Equality already permits a cutoff.
B has an unseen leaf 0	Its actual value is 0, though this search skipped that leaf.
Correct root policy	Keep A. Do not replace it with B merely because the returned bound is 5.

Alpha-beta preserves the optimal root value. Tie handling must respect bound semantics.

Earlier strong alternatives create earlier cutoffs

At MAX

Promising high-value actions first.

At MIN

Promising low-value replies first.

Practical hints

Use previous search results, forcing moves or domain-specific scores.

Include the cost of ordering in a performance comparison.

The true minimax values are generally unknown before searching.

The same tree, three exploration orders

Traversal

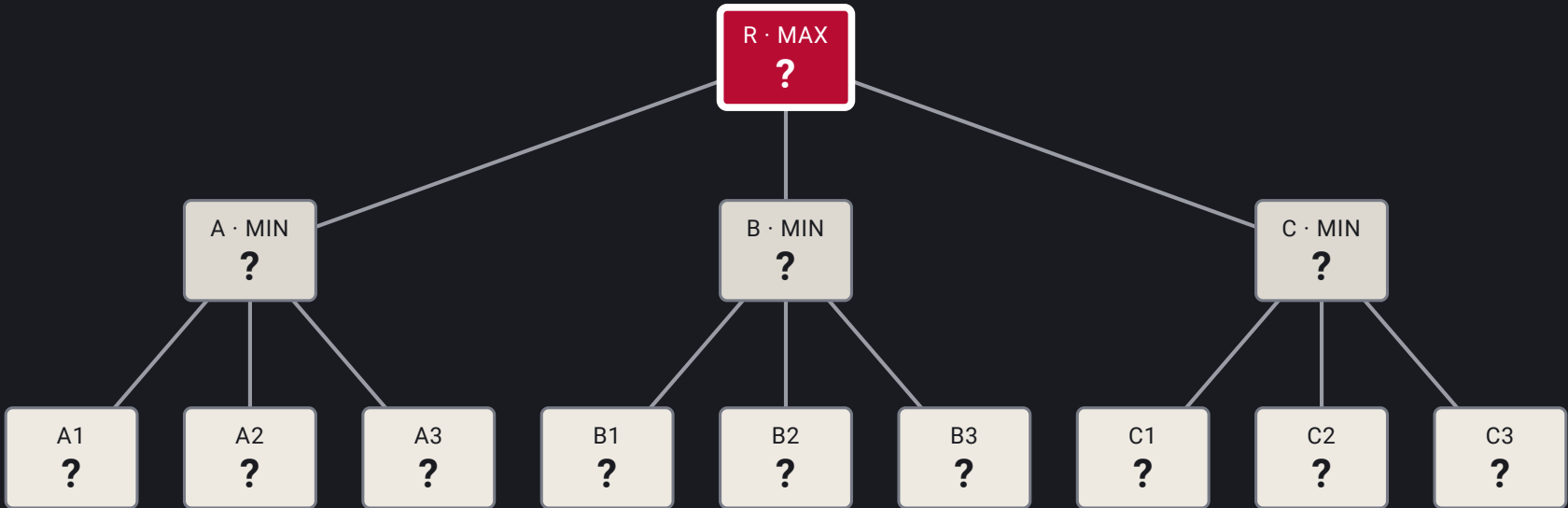
Listed order

Back

Next event

To result

Reset



Enter R (MAX).

Leaves read

0/9

Window

$[-\infty, \infty]$

A: 4,11,7 · B: 1,8,9 · C: 6,3,10. Oracle order uses known full values for teaching; it is not a free search heuristic.

Move ordering changes the number of evaluated leaves

Method / order	Evaluated leaves	Skipped leaves	Root result
Full minimax	9	0	A, value 4
Alpha-beta, listed order	6	3	A, value 4
Alpha-beta, oracle strong first	5	4	A, value 4
Alpha-beta, oracle weak first	9	0	A, value 4

These counts explain this tree. They do not establish a universal runtime ranking.

Ordering determines how much of the tree survives

Worst case

$O(b^m)$

Pruning may save little or nothing.

Ideal ordering

about $O(b^{m/2})$

The familiar even-depth model can roughly double depth for the same leaf budget.

The precise best-case leaf count in a uniform depth- m tree is $b^{\lceil m/2 \rceil} + b^{\lfloor m/2 \rfloor} - 1$.

Iterative deepening keeps a completed decision available

01

Depth 1

Finish a shallow search and keep its move.

02

→ Depth 2, 3, ...

Use earlier results to order the next search.

03

→ Budget ends

Return the last completed iteration's decision.

A partly searched deeper iteration does not automatically provide a better justified move.

A transposition table stores evidence about a state

Stored field	Why it matters
State key	Includes player to move and relevant rule state.
Search depth	States how much continuation the entry covers.
Value and type	EXACT, LOWER bound or UPPER bound.
Best known action	Useful for move ordering even when the value cannot be reused.

Compare a fail-soft result with the original window: $\leq \alpha_0$ gives UPPER; $\geq \beta_0$ gives LOWER; inside gives EXACT.

Tic-tac-toe supplies a complete testable game

1	2	3
4	5	6
7	8	9

Legal actions

Place the current player's mark in any empty square.

Terminal states

A completed row, column or diagonal, or a full board without a winner.

Fixed viewpoint

X is MAX (+100). O is MIN (−100). A draw is 0.

Play against an exact minimax opponent

1	2	3
4	5	6
7	8	9

You play X. Select an empty square. O responds using full minimax.

New game

Play best X move

Undo round

Your turn: X. O will minimize X's exact outcome.

Exact value of the current position for X

0

Last actions

No moves yet.

With optimal play from both sides, the empty board has value 0. A human mistake can give O a forced win.

Exhaustive enumeration can check every reachable position

Reachable states	5,478 board positions, stopping immediately after a win.
Empty-board value	0 under optimal play.
Validation	Check legal actions, terminal outcomes and optimal decisions at every state.
Scope	This finite result does not establish performance on a larger game.

Count states separately from move sequences. Different histories can reach the same board.

Negamax changes the value convention

Fixed MAX viewpoint

MAX takes a maximum.
MIN takes a minimum.

Current-player viewpoint

In a zero-sum alternating game, negate the child's value and always maximize.

$$N(s) = \max_a -N(\text{Result}(s,a))$$

Terminal and cutoff values must use the same current-player convention.

A random outcome uses a weighted average

MIN chooses

Use the lowest child value when a strategic opponent controls the decision.

Chance samples

Use the probabilities supplied by the model.

$$V = \sum_i p_i V_i$$

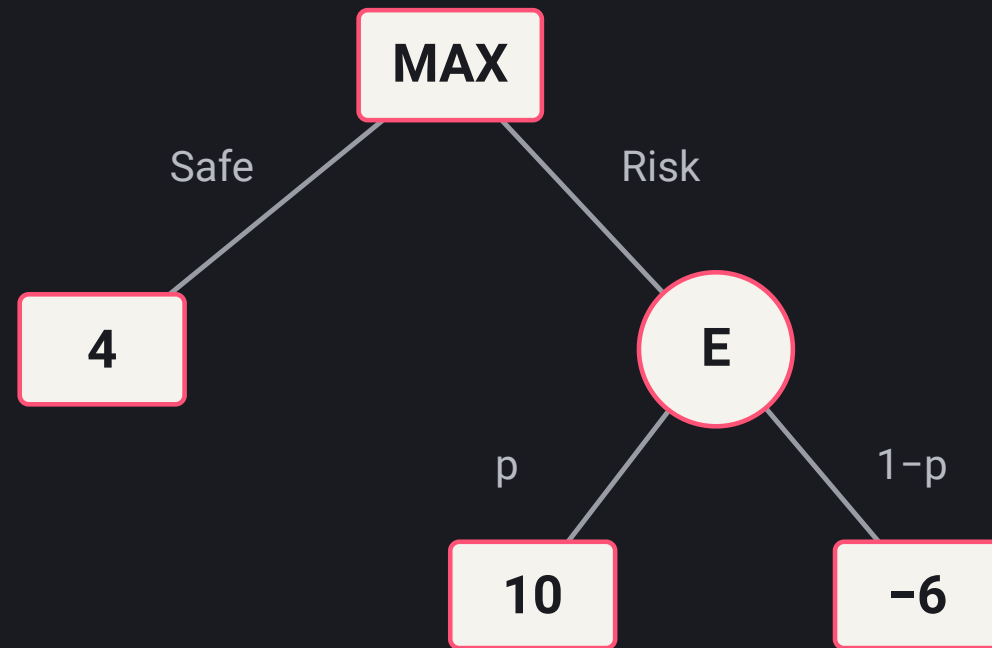
Require $p_i \geq 0$ and $\sum_i p_i = 1$. The probabilities need a justified source.

MAX, MIN and CHANCE can appear in one model

Node type	Backup rule	Small example
MAX	Largest child value.	$\max(4, 5.2) = 5.2$
MIN	Smallest child value.	$\min(5.2, 6) = 5.2$
CHANCE	Probability-weighted sum.	$0.7 \times 10 + 0.3 \times (-6) = 5.2$

The node type comes from the game’s transition rules; levels need not repeat in a fixed pattern.

A probability can reverse the selected action



Probability of +10: 0.5



Risk: $10p - 6(1-p)$

2

Expected-utility decision

Safe

$p=0.5$

$p=0.625$

$p=0.75$

Safe has the larger expected utility at this probability.

This is the MAX/CHANCE special case of expectiminimax. All outcomes remain specified even at probability 0.

The switch occurs at $p = 0.625$

Probability p	Expected Risk value	Preferred action
0.50	$0.5 \times 10 + 0.5 \times (-6) = 2$	Safe, value 4.
0.625	$0.625 \times 10 + 0.375 \times (-6) = 4$	Tie: Safe or Risk.
0.75	$0.75 \times 10 + 0.25 \times (-6) = 6$	Risk, expected value 6.

16p – 6 > 4 precisely when $p > 10/16 = 0.625$.

Averaging uses more than an ordering of outcomes

Scale at p=0.5	Safe	Expected Risk	Choice
Original utilities	4	$(10-6)/2 = 2$	Safe
Cubed utilities	64	$(1000-216)/2 = 392$	Risk

A strictly increasing transformation preserves deterministic minimax choices, but can change expected-utility choices.

A partial average needs bounds on the unseen outcomes

Visited contribution

S = sum of $p_i V_i$ for evaluated outcomes.

r = remaining probability mass.

If every unseen value lies in $[L, U]$

$$S + rL \leq V \leq S + rU$$

Only a justified interval can support an early decision.

Ordinary MIN/MAX alpha-beta cutoffs cannot simply be copied onto an arbitrary chance node.

A policy must respect what the player can observe

One observed situation

Several hidden states can look identical to the acting player.

The policy must choose using the available information.

A modelling error

Solving each hidden state separately and selecting a different action can use information the player does not have.

Beliefs and information sets replace the assumption that the full current state is visible.

AlphaZero combines learned estimates with lookahead

01

Self-play

Generate games using the current system.

02

→ Learn

Fit policy and value predictions from play.

03

→ Search

Use those predictions to guide Monte Carlo tree search.

A learned value function and a search algorithm can improve one another.

A reliable game-search interface has clear contracts

State transition	Generate legal moves and preserve the input state when exploring a child.
Perspective	Use one declared value convention in terminal and heuristic code.
Search control	Check terminal states, count plies and keep a legal fallback under limits.
Result type	Return action, score, depth and the meaning of any bound.

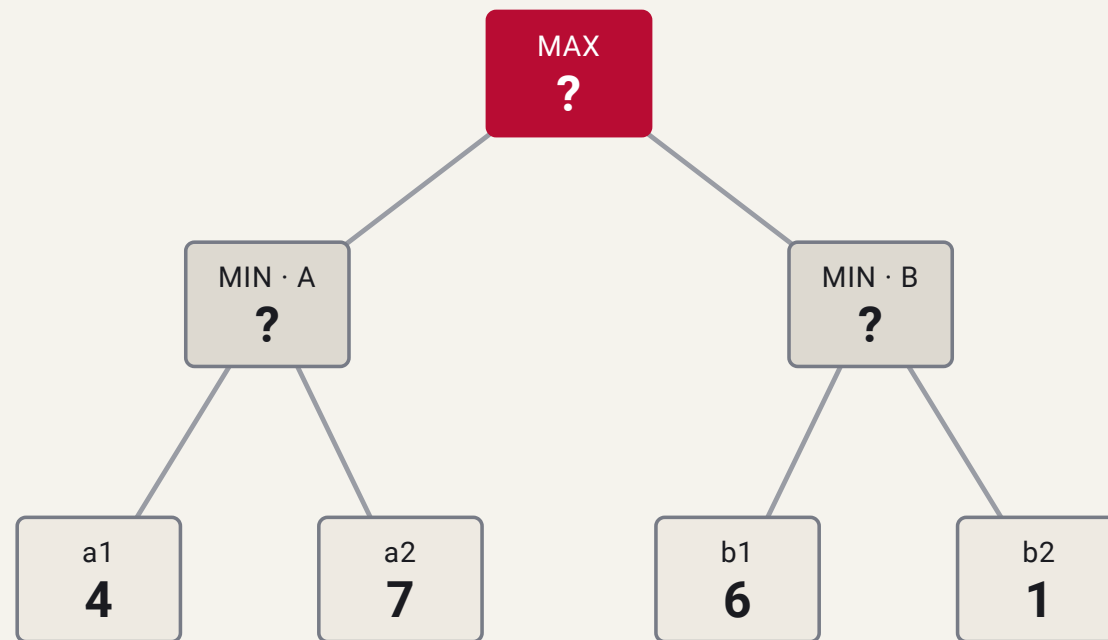
A correct numeric backup cannot rescue an incorrect game model.

Small examples provide exact expected answers

Test	Expected evidence
Source minimax tree 3,12,8,2	Root value 3, action A.
Pruning tree 5,6,2,9	Root 5, action A; 3 evaluated leaves.
Tie tree 5,6,5,0	Root 5, keep A; B returns only an upper bound.
Source tic-tac-toe board	Terminal X win; no further legal continuation.
Empty tic-tac-toe board	Exact value 0.
Chance $p=0.625$	Both actions have expected utility 4.

Compare alpha-beta with exhaustive minimax on many small trees, including ties and negative values.

Minimax backup with the source exercise

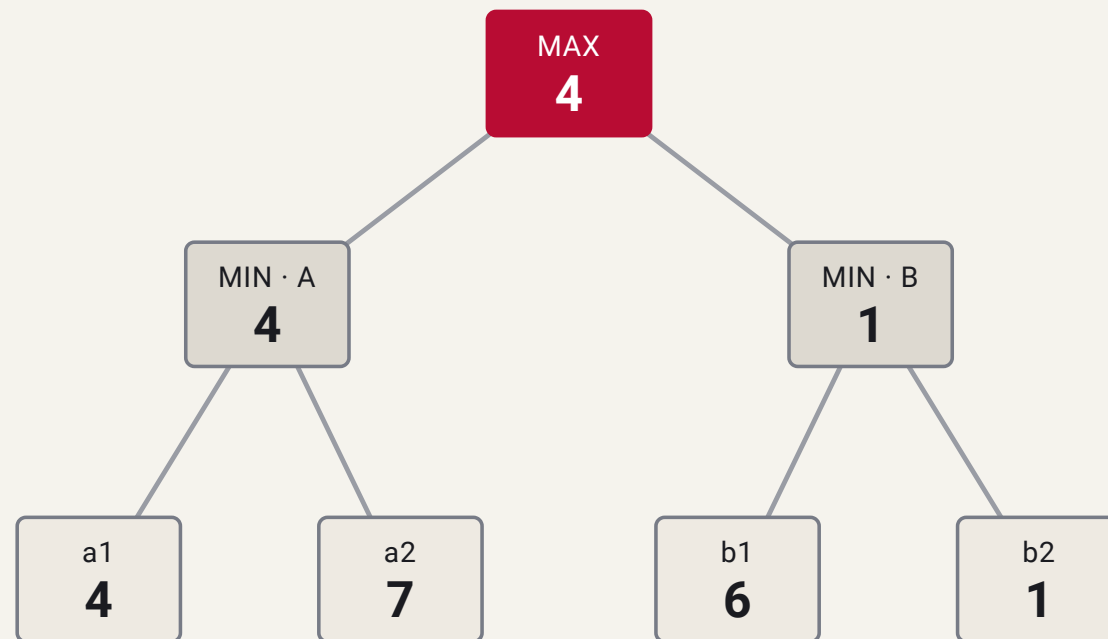


5 min

1. Compute A and B.
2. Compute the root value.
3. State the action.
4. Explain why the leaf value 7 is not guaranteed.

The leaves are terminal utilities for MAX. MIN controls both second-level nodes.

The root chooses A with guaranteed value 4



MIN backups

$$A = \min(4, 7) = 4.$$

$$B = \min(6, 1) = 1.$$

MAX decision

$$\max(4, 1) = 4.$$

Choose A.

MIN can choose 4 instead of 7.

Pruning depends on the order and on what is still unknown

Case	Task
A=(5,6), B=(2,9)	Run alpha-beta left to right. List evaluated and skipped leaves.
A=(5,6), B=(9,2)	Repeat after reversing only B's leaf order.
A=(5,6), B=(5,0)	Explain the equality cutoff and which root action must be retained.

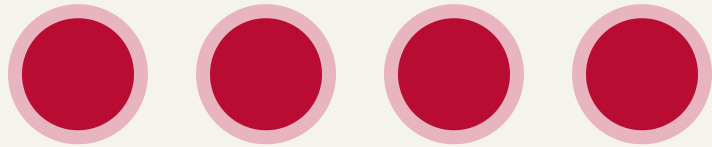
Report the root value, chosen action and whether the returned B score is exact or only a bound.

Three traces, one correct root action

B leaf order	Leaves evaluated	Skipped	Knowledge about B
2,9	5,6,2	9	$B \leq 2$ from the search.
9,2	5,6,9,2	None	$B = 2$ after both leaves.
5,0	5,6,5	0	$B \leq 5$ from the search; actual $B = 0$.

All three return A with root value 5. Equal bounds must not replace an established exact best action.

A take-away game with four tokens



8 min

On each turn remove 1 or 2 tokens.
The player taking the last token wins.

Specify the state

Include tokens remaining and player to move.

Define the rules

Actions, transition, termination and MAX utility.

Evaluate MAX's opening

Compare removing 1 with removing 2.

Explain a strategy

What response preserves a win after MIN moves?

This is normal play. Changing the last-token rule creates a different game.

Leaving three tokens gives MAX a winning strategy

MAX opening	MIN's possible continuation	Result under optimal play
Remove 1 → leave 3	MIN removes 1: leave 2; MAX takes 2.	MAX wins.
Remove 1 → leave 3	MIN removes 2: leave 1; MAX takes 1.	MAX wins.
Remove 2 → leave 2	MIN takes 2 immediately.	MIN wins.

Choose removal 1. Thereafter make each MIN/MAX pair of removals sum to 3.

Six distinctions that prevent search errors

01

Why does MIN take a minimum?

Values use MAX's perspective, and the zero-sum opponent prefers the least favourable outcome for MAX.

02

What does a cutoff score prove?

It is a result for the truncated tree and chosen evaluator, not generally the exact full-game value.

03

When can alpha-beta stop?

When $\alpha \geq \beta$ makes the remaining children irrelevant to the established root decision.

04

Can a pruned node return a bound?

Yes. A MAX cutoff gives a lower bound; a MIN cutoff gives an upper bound.

05

Does ordering change the optimal value?

For the same tree and leaf values, no. It changes work and can change the selected tied optimal action.

06

Why average at chance nodes?

The model assigns probabilities to outcomes. Expected utility combines their values with those weights.

Sources and further reading

Supplied UCLM course material · 44 slides

01_Presentacion_Tema5_Adversarial_Search(3).pptx.

Scope, numerical minimax exercises and the tic-tac-toe illustration.

Russell and Norvig · Artificial Intelligence: A Modern Approach

Official textbook resources · Adversarial Search and Games.

UC Berkeley · CS188

Minimax and alpha-beta · **Expectimax and mixed node types**

Silver, Hubert, Schrittwieser and Hassabis · 2018

Official account of AlphaZero's search and self-play learning

A value estimate can be learned from experience

Unit 5

A model predicts continuations.
Search compares the resulting values.

look ahead · evaluate · act

Unit 6

Interaction produces experience.
Rewards help improve future decisions.

experience · reward · learning

Next: reinforcement learning, value functions and the exploration–exploitation tradeoff.